

Average-Case Analysis (and Further Analysis Practice)

Design and Analysis of Algorithms

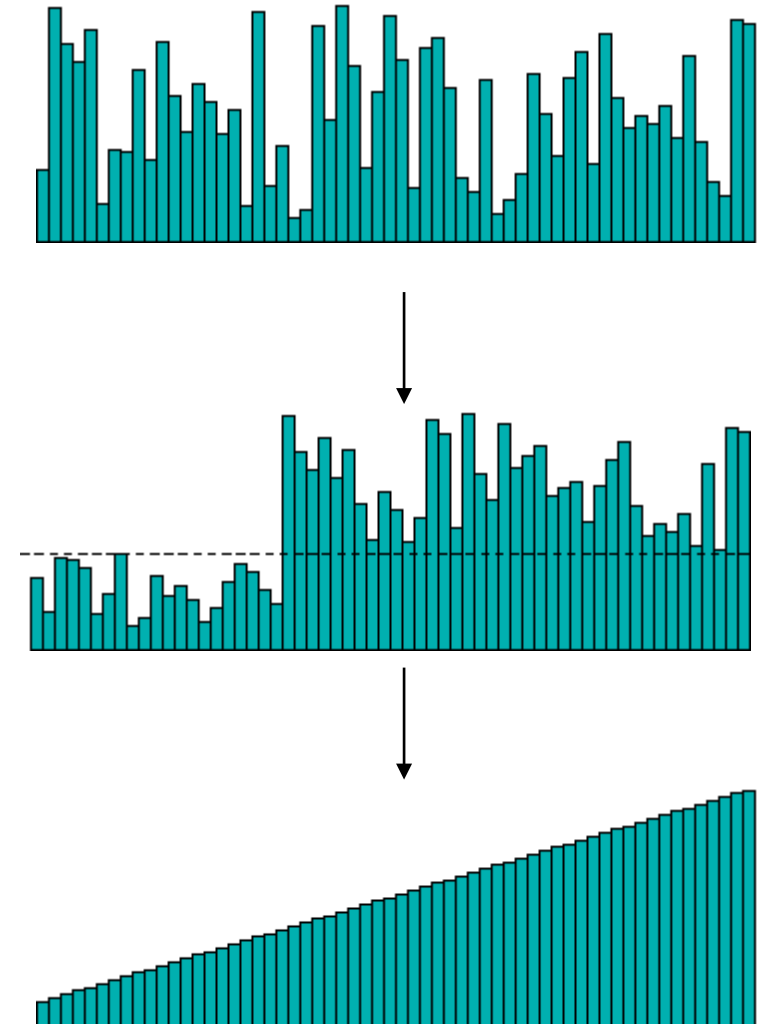
Brian C. Dean

**School of Computing
Clemson University**



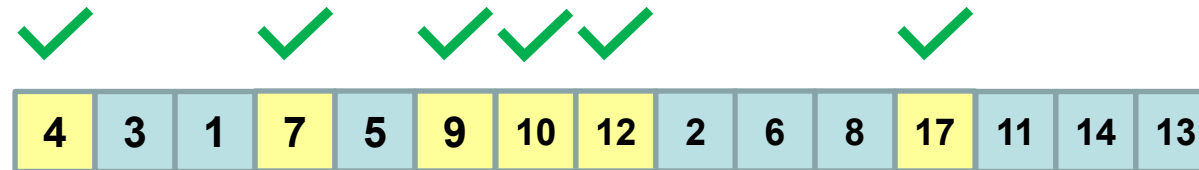
Expected Running Time of Randomized Algorithm Versus Average Case

1. Randomized quicksort: $O(N \log N)$ expected time.
2. Deterministic quicksort where we simply choose pivot to be 1st element of array: ill-advised; $\Theta(N^2)$ if applied to already-sorted array.
3. Running (2) after first scrambling array: ok, since essentially (1).



Warm-Up: Resetting a Running Max

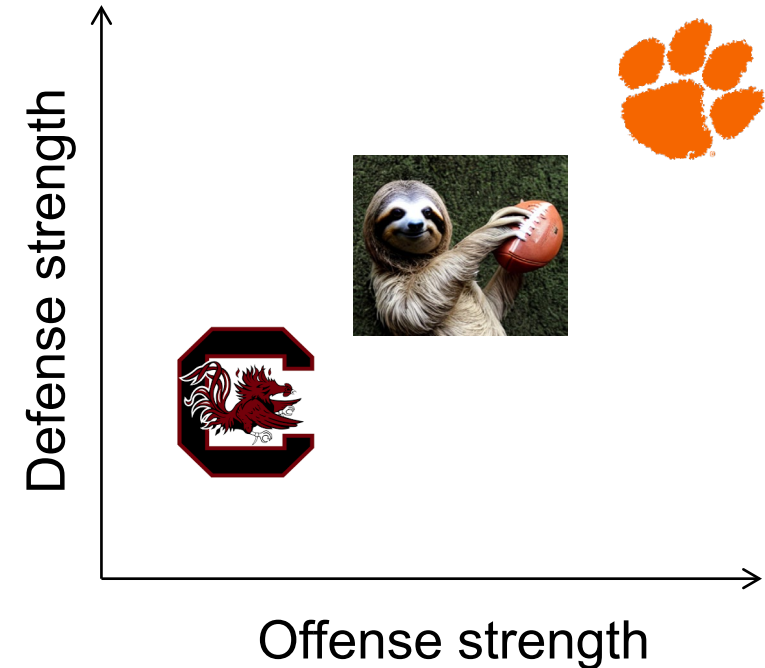
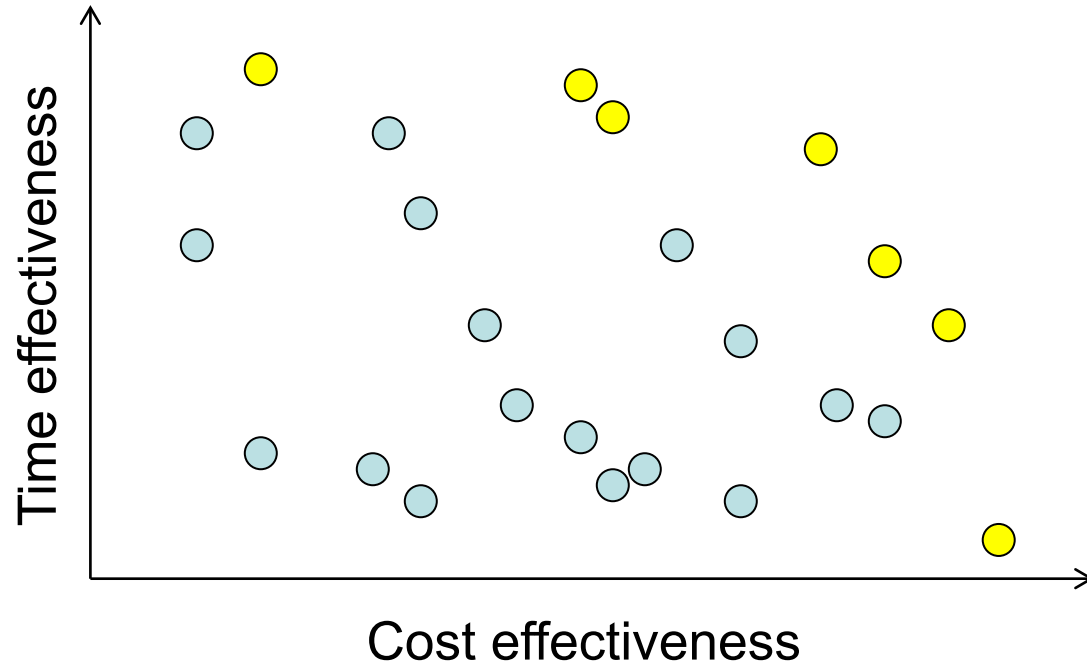
- Randomly permute a length-N array of distinct values.
- Scan the array and maintain a running max.
- Show that you expect to reset the max $O(\log N)$ times.



- Let's analyze this two different ways...

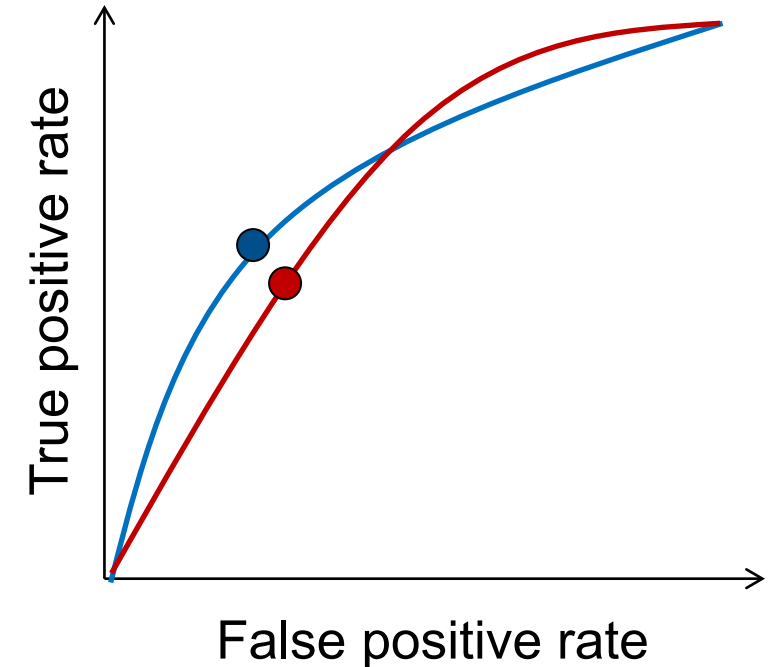
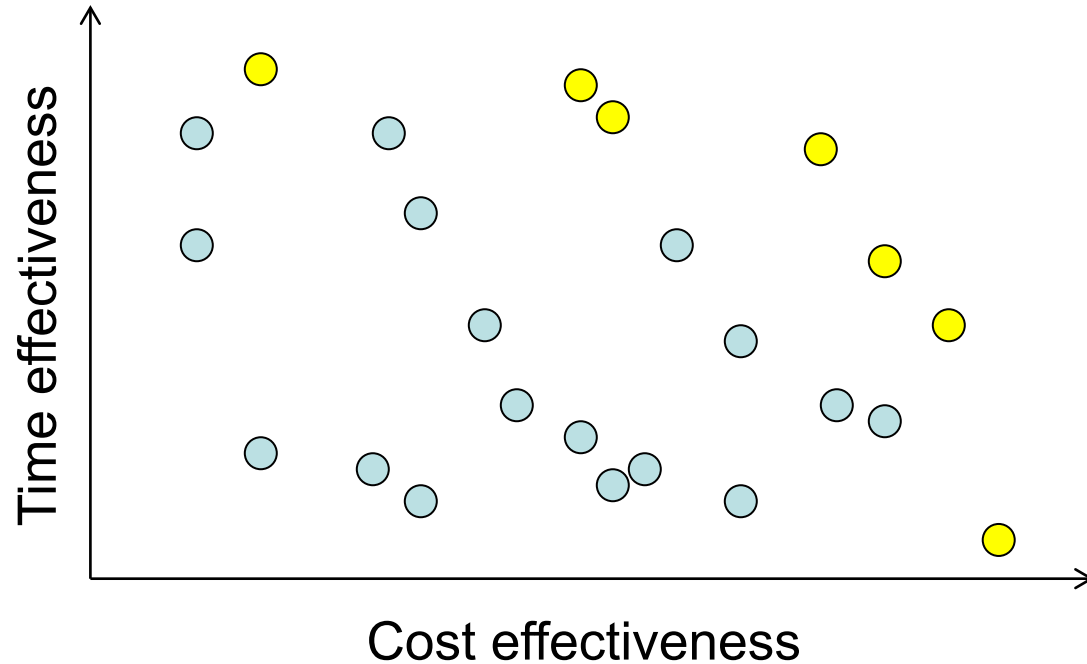
Example: Multi-Objective Optimization

How many “non-dominated” or “Pareto-optimal” solutions to you expect with N random 2D points?

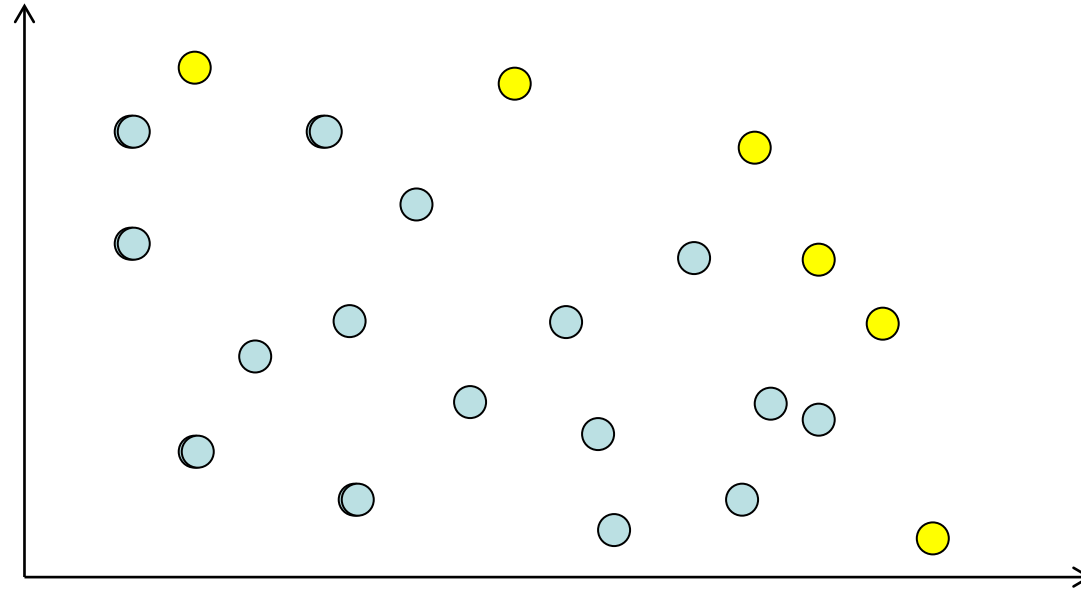


Example: Multi-Objective Optimization

How many “non-dominated” or “Pareto-optimal” solutions to you expect with N random 2D points?



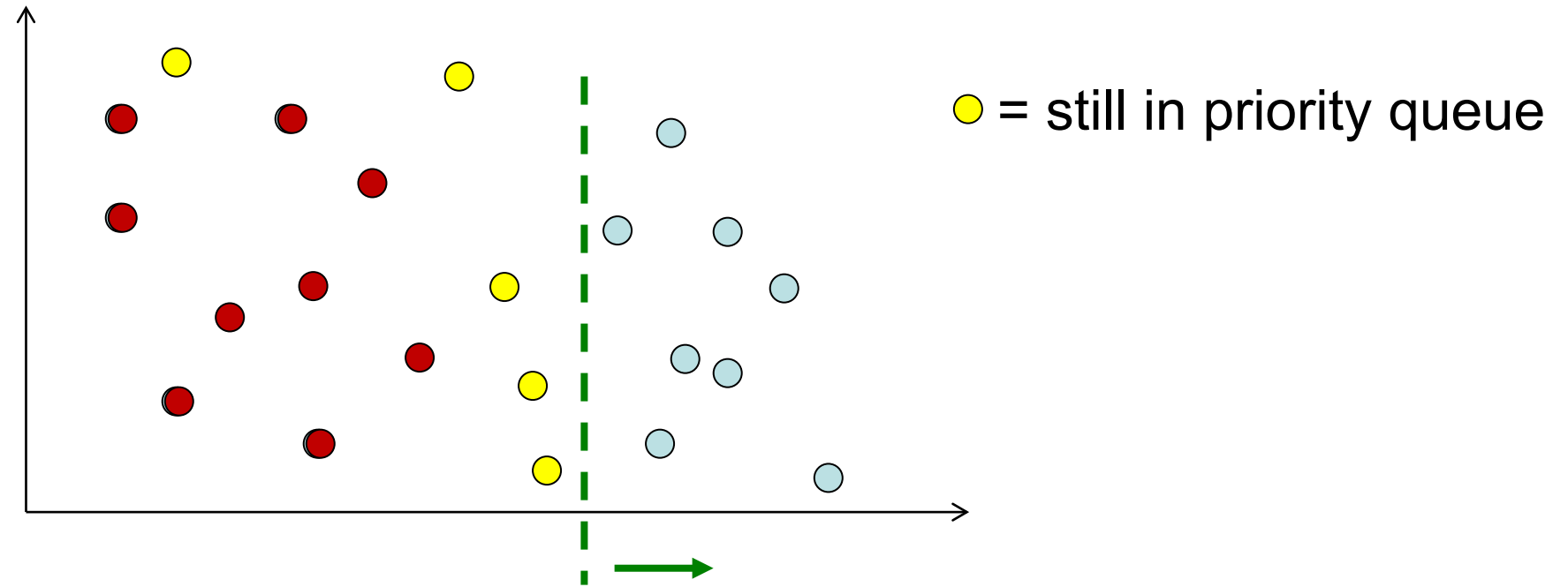
Finding Non-Dominated Points



How would you efficiently identify all the non-dominated points?

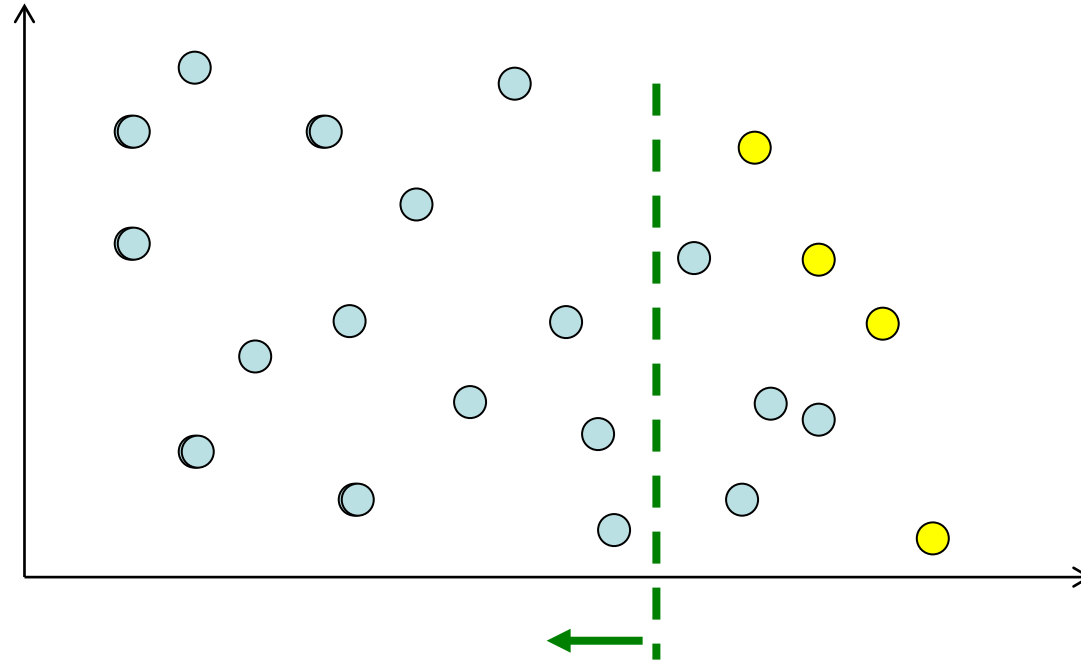
(just in 2D... like many other geometry problems, this problem suffers from the so-called “curse of dimensionality” and seems progressively harder to solve efficiently in higher dimensions...)

Non-Dominated Points: “Sort + Scan” / Incremental Construction



- Sort on x (in $O(N \log N)$ time with merge sort or quicksort)
- Scan points in increasing order of x. Store in priority queue on y.
- N inserts, $\leq N$ remove-mins: $O(N \log N)$ total!

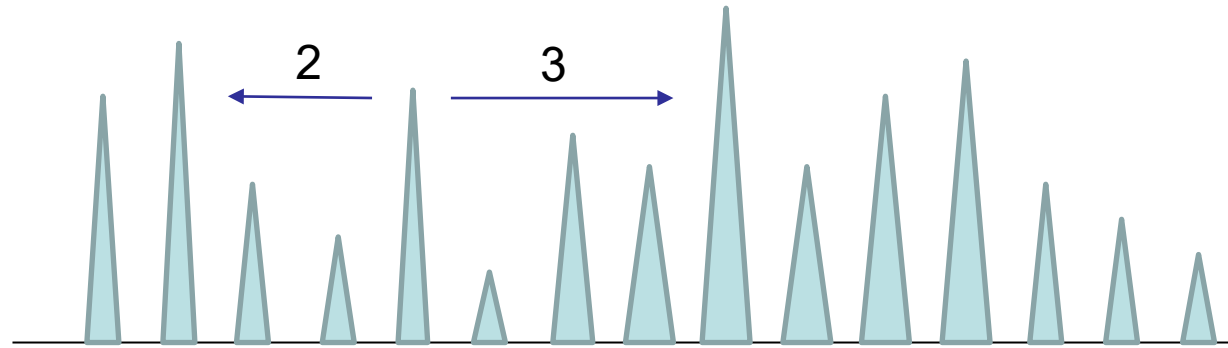
Non-Dominated Points: “Sort + Scan”



- Scan in decreasing order of x . Keep running max on y .
- Also $O(N \log N)$ total, due to initial sort on x .
- Random points $\rightarrow O(\log N)$ expected resets of running max!

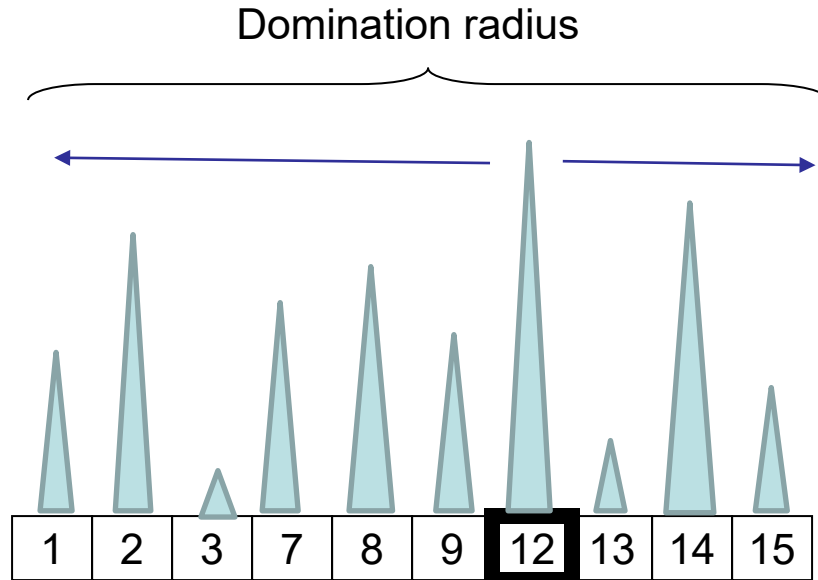
Exploiting Connections Between Problems and Algorithms

Remember the “domination radius” problem from some of our mini-lectures in the previous module?

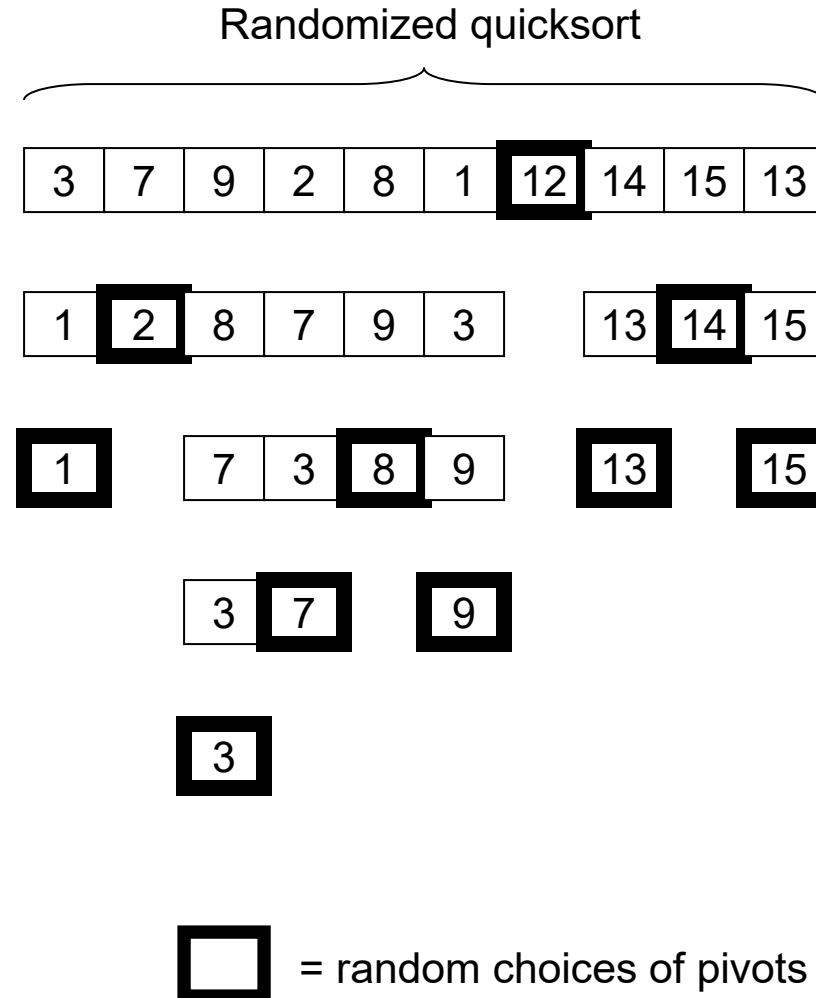


The running time is $\Theta(N^2)$ in the worst case if you scan left and then right, but what happens “in the average case” if you apply this algorithm to a randomly-permuted array?

Correspondence With Randomized QuickSort!

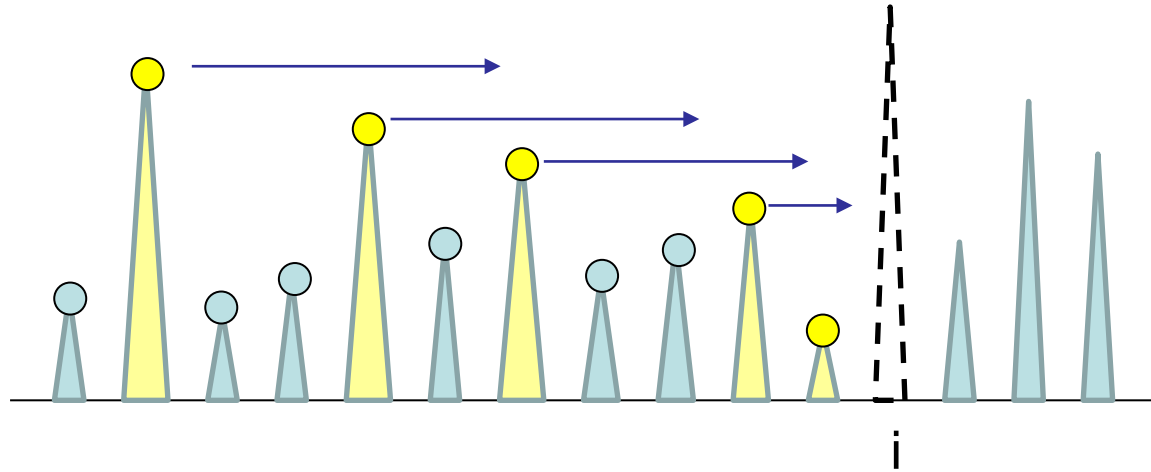


Height corresponds to how early an element was chosen as a pivot (tallest chosen first).



Resetting a Running Max Again...

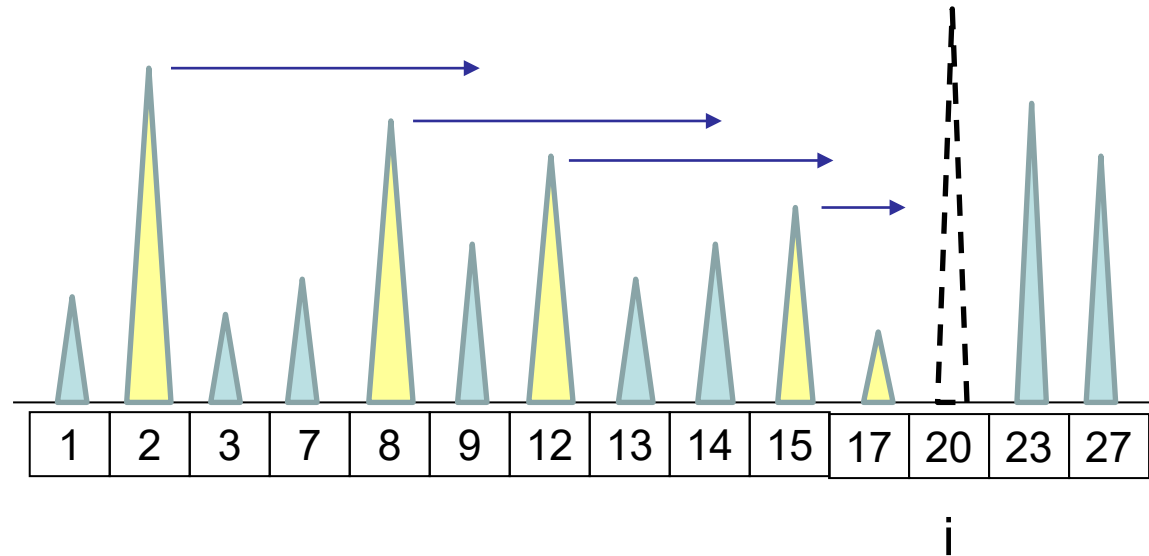
Scanning outward, only $O(\log N)$ “interesting” elements expected on each side...



Only these will compare themselves to the element at position i .

Mapping Back to Randomized Quicksort

Scanning outward, only $O(\log N)$ interesting pivots expected on each side.

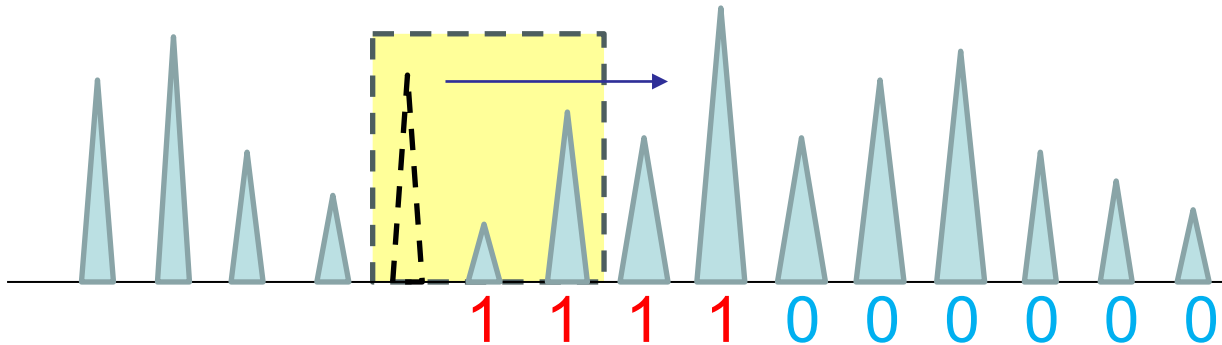


Only these will compare themselves to the element at position i .

Even More Ways to Analyze...

Expected Scan Distance

Using linearity of expectation: $E[\text{scan distance to right}] = E[X_1] + E[X_2] + E[X_3] + \dots = 1/1 + 1/2 + 1/3 + \dots = O(\log N)$



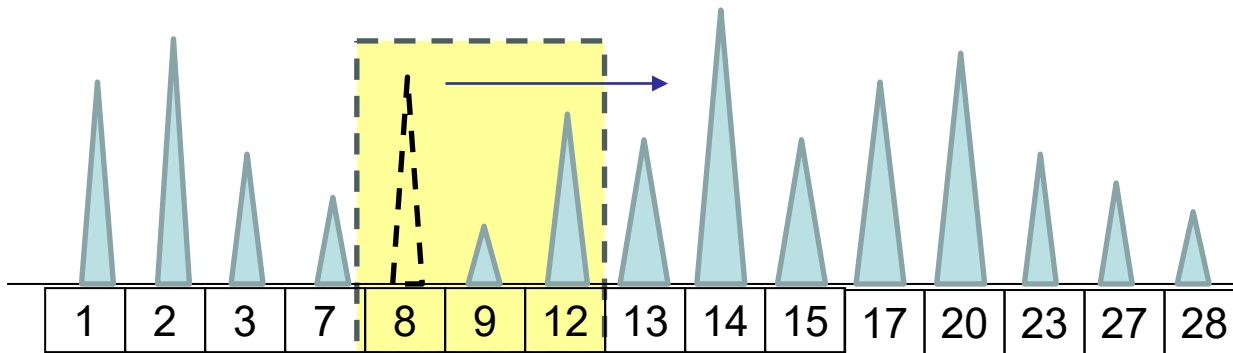
$X_1 = 1$ if scan right at least distance of 1. $E[X_1] = 1/1$.

$X_2 = 1$ if scan right at least distance of 2. $E[X_2] = 1/2$.

$X_3 = 1$ if scan right at least distance of 3. $E[X_3] = 1/3$.

Mapping Back to Randomized Quicksort

Using linearity of expectation: $E[\# \text{ comparisons on right}] = E[X_1] + E[X_2] + E[X_3] + \dots = 1/1 + 1/2 + 1/3 + \dots = O(\log N)$



Height corresponds to how early an element was chosen as a pivot (tallest chosen first).

$X_1 = 1$ if at least 1. $E[X_1] = 1/1.$

$X_2 = 1$ if at least 2. $E[X_2] = 1/2.$

$X_3 = 1$ if at least 3. $E[X_3] = 1/3.$

We'll Return to These Pictures When Studying Binary Search Trees...

